FVM: Practical Feather-Weight Virtualization on Commodity Microcontrollers

Junchao Li, Runsheng Hou, Guangyong Shang, Huanle Zhang, Xiuzhen Cheng, *Fellow, IEEE*, and Runyu Pan

***Abstract*—Recently, there has been an increasing drive to consolidate multiple microcontrollers into one physical entity, due to advantages in reducing overall costs, enhancing reliability, and simplifying hardware interconnections. To reduce consolidation engineering costs, minimizing system latency and memory footprint is important as well as maintaining compatibility with legacy software. In this paper, we propose a virtualization-based solution called Feather-weight Virtual Machine (*FVM*) that focuses on these goals. *FVM* enables low latency by specializing the virtualization model to Real-Time Operating Systems (RTOSes), achieves small footprint by adapting management policies to microcontroller memories, attains high compatibility by aligning with microcontroller ecosystem idiosyncrasies, finally allowing practical consolidation across a wide range of commodity microcontrollers. We implement and evaluate *FVM* on ARMv6-M, ARMv7-M, and RISC-V architectures with two toolchains and two RTOSes, and it can fit into 20 KiB of RAM with less than 5% latency bloat.**

***Index Terms*—Microcontroller consolidation, virtualization, microkernel.**

I. Introduction

MICROCONTROLLERS are minicomputers that pack all essential hardware – including a CPU, memory, and peripherals – onto a single chip that costs less than $10. They are ubiquitous in automotive, industrial, and consumer products, where they bring intelligence into machines. Typically designed to perform specific tasks, microcontrollers run firmware as a single binary image that boots in bare-metal mode, lacking runtime protection domain isolation. Resource scarcity is often used as a scapegoat for this lack of isolation, as they face stringent restrictions in terms of Size, Weight, Power, and Cost (SWaP-C).

As product complexity increases, more microcontrollers are needed, and they are interconnected through communication interfaces *i.e.* CAN, SPI and IIC. However, the growing complexity of interconnections has led to cost and reliability challenges [1], and the need to consolidate microcontrollers has taken off [2].

The mainstream approach in microcontroller consolidation is to rewrite all legacy firmware and developing an integrated bare-metal image. However, this approach faces three difficulties. First, rewriting legacy code requires substantial effort and forfeits previous certifications. Second, the legacy firmware may rely on incompatible RTOSes or frameworks that are challenging to coexist. Last, the integrated bare-metal image often lacks internal isolation, which can lead to fault or compromise propagation [3], [4].

In the cloud industry, virtualization is broadly employed to perform consolidation without breaking legacy compatibility and isolation [5], [6]. In the microcontroller arena, virtualization is also being explored [7], [8], where each Virtual Machine (VM) corresponds to a virtual microcontroller. However, several drawbacks prevents the practical use of virtualization for microcontroller consolidation [7], [9], [10], including: (1) high execution overheads, (2) significant memory footprint, (3) need for specialized hardware, and (4) incompatibility with existing ecosystems.

This paper introduces *FVM*, a lightweight type II[1] virtualization framework for microcontroller consolidation. As Fig. 1 illustrates, *FVM* integrates VMs, native processes, a hypervisor and a microkernel. Specifically, *FVM* (1) minimizes latency by specializing the virtualization model for RTOSes, (2) reduces footprint by adapting memory management policies to microcontroller memory constraints, (3) maximizes compatibility by aligning with the idiosyncrasies of the microcontroller ecosystem, and (4) enables practical consolidation across a wide range of commodity microcontrollers.

Contributions. The primary contribution of this paper is a demonstration showing that virtualization can be effectively

[1]Type I hypervisor runs directly on the bare-metal hardware, while type II hypervisor runs alongside a host OS that provides basic services.

Received 15 December 2024; revised 11 March 2025; accepted 31 March 2025. Date of publication 8 April 2025; date of current version 11 June 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4502001; in part by the National Natural Science Foundation of China under Grant 62402291, Grant 62302265, and Grant U23A20332; and in part by Shandong Province Natural Science Foundation under Grant ZR2023QF172 and Grant 2024HWYQ-020. Recommended for acceptance by Y. Hu. *(Junchao Li and Runsheng Hou contributed equally to this work.) (Corresponding author: Runyu Pan.)*

Junchao Li, Runsheng Hou, Huanle Zhang, Xiuzhen Cheng, and Runyu Pan are with the School of Computer Science and Technology, Shandong University, Qingdao 266237, China (e-mail: junchaoli@mail.sdu.edu.cn; rhou@mail.sdu.edu.cn; dtczhang@sdu.edu.cn; xzcheng@sdu.edu.cn; rypan@sdu.edu.cn).

Guangyong Shang is with the School of Computer Science and Technology, Shandong University, Qingdao 266237, China, and also with the Inspur Yunzhou Industrial Internet Company Ltd., Jinan 250101, China (e-mail: shangguangyong@inspur.com).

Digital Object Identifier 10.1109/TC.2025.3558582

0018-9340 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

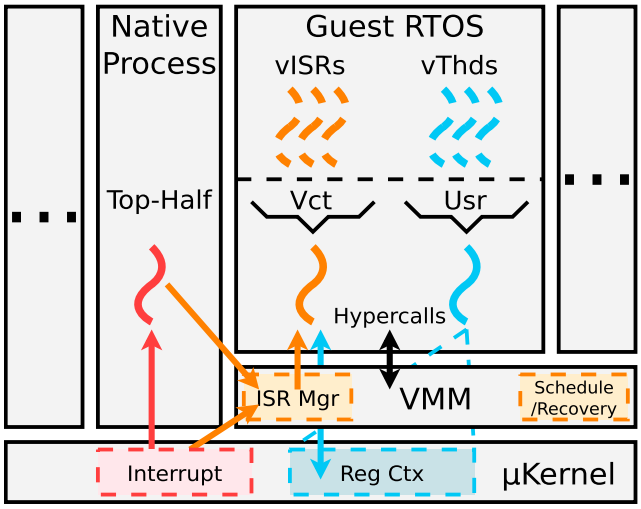

Fig. 1. *FVM* overview. The dashed curves represent guest VM threads (user-level threads) while the solid curves represent microkernel threads (kernel-level threads). The red and orange arrows indicate the fast- and regular-path for interrupt delivery respectively, and the blue double arrows represent that the VCT can modify USR thread kernel-level context directly without microkernel system calls.

applied for consolidation purposes, even on extremely resource-constrained commodity microcontrollers.

The paper offers the following contributions:

- **we detail the *FVM* design** (Section III) that enables state-of-the-art high-performance, low-footprint practical virtualization-based consolidation on commodity microcontrollers;
- **we perform *FVM* implementations** (Section IV) on four commodity microcontrollers, three popular architectures, two open-source RTOSes and two industry-standard toolchains, thereby showcasing its wide compatibility and portability;
- **we perform a comprehensive *FVM* evaluation** (Section V) across multiple hardware/software combinations, demonstrating its low overheads (as little as 5% execution time) and small footprint (as low as 20KiB).

To the best of our knowledge, *FVM* is the first framework to deliver practical performance while ensuring reliability, security, and compatibility across a wide range of commodity microcontroller consolidation scenarios.

II. Background and Related Work

A. Memory Protection Facilities

Modern microcontrollers provide memory isolation through Memory Protection Units (MPUs). Unlike Memory Management Units (MMUs), MPUs cannot translate virtual addresses to physical addresses, and only allow dividing the physical address space into regions with varying access permissions. While MPUs have limited regions and cannot perform address translation, they do not generate TLB misses and are more predictable.

Numerous works have leveraged MPUs to provide isolation on microcontrollers. *FreeRTOS*-MPU [11] restricts memory accesses with MPU, but only addresses safety concerns without considering security [12]. Safer Sloth [13] swaps protection domain between thread switches, which guards against transient faults. TockOS [14] relies on Rust for kernel safety and relies on MPU to confine other applications. ACES [15], [16] divides the program into compartments according to security policies and segregates them with MPUs. uXOM [17] implements execute-only memory with MPUs and unprivileged memory access instructions. Pip-MPU [18] implements a formally verified separation kernel with MPUs. Other works, such as Mbed µVisor [19], uSFI [20] and OPEC [21] also use MPU to confine their protection domains. Compared to virtualization, they require new languages, toolchain modifications or workflow changes that are incompatible with the microcontroller ecosystem.

In addition, some works have explored virtualization [7], [10], [22], [8]. However, they have high execution overheads and large memory footprints that is often many times that of the original RTOS, which makes them impractical for consolidation scenarios.

B. Software Fault Isolation (SFI)

Many works bound memory accesses with software constructs, achieving the same as hardware MPUs. These works can be further divided into two categories.

The first category relied on compiler checks or binary verification. ARMor [23] leverages binary verification to isolate faults. CRT-C [24] formalizes program compartments and achieves privilege separation with specialized language dialects. Minion [25] and TrustLight [26] determine memory compartments offline and enforce isolation with the help of custom hardware.

The second category leverages managed execution environments originally designed for resource-rich environments, also known as bytecode interpreters. Recently, many of them are ported to microcontrollers, with the most prominent ones being eBPF [27] and Python [28].

However, both are unsuitable for microcontroller consolidation, because they (1) may restrict the choice of programming languages, (2) cannot be linked against third-party binaries without breaking isolation, and (3) induce long latency and large memory overheads when compared to the native binaries.

C. Control Flow Integrity (CFI)

Some works enforce CFI instead of or in addition to isolation, through runtime software or hardware checks, thereby defeating control-flow attacks. RECFISH [29] inserts checks on function calls/returns and maintains a separate MPU-protected return stack. WARduino [30] and aWSM [31] leverage WebAssembly sandbox to provide control- and partial data-flow integrity, and the latter leverages MPU for bounds checking. µRAI [32] stores a list of possible return addresses in Flash memory and verifies returns against them. CaRE [33] leverages *ARMv8-M TrustZone* to accelerate shadow stack operations, whereas Kage [34] and SuM [35] achieve the same with MPU and unprivileged memory access instructions. Sherloc [36] exploits hardware tracing buffers that were intended for debugging instead of a shadow stack.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

However, their applicability in microcontroller consolidation is limited, because they (1) sometimes do not provide isolation, (2) insert run-time checks that bloat execution latency, (3) cannot be linked against third-party binaries without breaking integrity, and (4) may require customized toolchains or hardware that is not always available in a production setting.

D. Microcontroller Ecosystem Idiosyncrasies

Different from regular application engineering practices, the microcontroller development ecosystems have the following idiosyncrasies: (1) the certified compilers like IAR [37] cannot be customized, (2) the compilers may generate proprietary or esoteric object formats, (3) the linkers may not support sophisticated object symbol manipulations, (4) the debuggers may only support the proprietary executable formats, (5) the toolchain always assume bare-metal, freestanding environments.

These idiosyncrasies render researches involving toolchain or workflow customization less useful in practice, and the *FVM* must therefore account for these factors and stay fully compatible with the current ecosystem.

III. *FVM* Design

The main goal of *FVM* is to enable practical virtualization-based consolidation on commodity microcontrollers. The detailed goals include:

G1: Low execution latency. To cope with low latency requirements of the original firmware, the system operations and interrupt responses in the VM must exhibit low overheads and high predictability.

G2: Small memory footprint. To make efficient use of limited memory, the data structures must be compact and the memory allocation must be flexible.

G3: Inter-VM isolation. To ensure security and reliability, the VMs must be confined *spatially* and *temporally* to stop the propagation of faults and compromises.

G4: Ecosystem compatibility. To be practical, the system must be compatible with the microcontroller ecosystem, including architectures, toolchains, RTOSes, workflows, and developer mental models.

A. Virtualization Architecture Overview

As shown in Fig. 1, we base the entire virtualization framework on a microkernel that hosts other components as processes. A type II hypervisor manages the guest VMs, within which two cooperating threads emulate a bare-metal vCPU. The lower-priority thread (Usr) is responsible for executing all tasks of the guest RTOS, while the higher-priority thread (Vct) handles interrupts and switches task context by modifying Usr's context.

Compared with a type I hypervisor design, this choice offers several benefits, including (1) a smaller Trusted Computing Base (TCB), (2) a cleaner separation of policy and mechanisms, and (3) supporting lighter-weight native processes in complement to VMs.

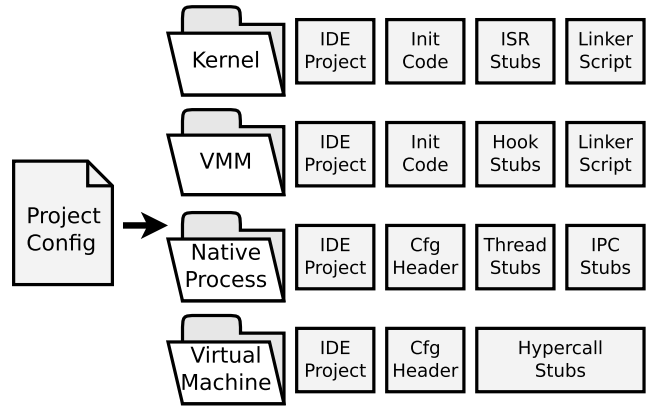

Fig. 2. *FVM* composer workflow. Only the major artifacts generated are shown.

To simplify consolidation workflow, a system composer shown in Fig. 2 generates the memory layout, boot-time initialization code, configuration headers and linker scripts. It also produces the Integrated Development Environment (IDE) projects to integrate seamlessly with the toolchain ecosystem.

B. Microcontroller-Specific Microkernel

To support *FVM*, we propose a capability-based microkernel tailored to the microcontroller architectures. Notably, it makes the following design decisions:

Exposing microkernel thread context to user-level. The context switches in RTOSes often employ interrupt vectors to manipulate the registers of the background execution. To eliminate the need for a dedicated system call [38] for such modification, we enable the kernel to save the Usr context inside the VM user space instead of the kernel space, so that the context can be modified by Vct directly **(G1)**. However, two security challenges must be overcome to prevent VM escapes: ensuring valid memory access and verifying sensitive register integrity **(G3)**. To this end, we (1) make sure the memory region falls within the physical memory to allow safe kernel access, and (2) insert checks on kernel context restore path to correct illegal sensitive register (*e.g. Cortex-M*'s LR or *RISC-V*'s mstatus) values.

Eliminating kernel object alignment requirements. To make efficient use of limited memory, the kernel objects allocation must be flexible and economical. Notably, MPUs do not perform virtual-to-physical address translation, but do not impose page size and alignment restrictions as Memory Management Units (MMUs) do, either. In these lights, we employ a fine-grained kernel memory allocator rather than a paged allocator, to capitalize on the absence of page size and alignment restrictions. It eliminates the artificial alignment constraints often imposed by other microkernels, minimizing external memory fragmentation **(G2)**. The kernel memories are not tracked in pages but rather in *kernel memory capabilities*, which corresponds to permission to create kernel objects in a memory segment. In this fashion, the kernel memory management policies are also exported to the user-level.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

Making in-kernel memory management optional. Given that microcontroller systems tend to statically allocate everything at boot-time, we forgo memory retyping and instead maintain a predetermined boundary between kernel and user memory **(G2)**. Furthermore, we observe that memory access permissions are also rarely modified at run-time. When this is the case, the system composer generates the MPU tables for all processes **(G2)** and store them in *read-only* flash memory **(G3)**, and instructs the kernel to exclude memory management once and for all. This further reduces the memory footprint as well as the attack surface of the kernel **(G3)**.

Simplifying in-kernel scheduling facilities. In the light of Mixed-Criticality Scheduling (MCS), modern microkernels abandon traditional threads for separate *scheduling* and *execution* contexts. Scheduling contexts hold budgets and policies, while execution contexts store thread register contents. This separation enables flexible policy and subsystem coordination with managed interference. However, the scheduler hot-path now involves both scheduling and execution context objects, increasing overhead for validity checks and data structure operations. Some systems [39] go even further to encode priority as a vector, storing all subsystem scheduler scalar priorities in the scheduling context to avoid priority flattening. In cases where priorities don't flatten, preemption decisions require comparing all subsystem priorities, adding more overheads. Though these overheads don't introduce substantial latency on high-performance processors where cache misses dominate latency, it does become a significant factor on microcontrollers that operate at a rate commensurate to their memory.

In *FVM*, we revert to the traditional thread concept and implement a FPRR policy in the kernel similar to most RTOSes. This policy is *not* a full scheduler: we keep room for bandwidth servers by removing implicit budget replenishes, and each subsystem scheduler now explicitly replenishes its threads' budgets, limiting the interference. If the priorities can be flattened to match the in-kernel FPRR, latencies are minimal, which is often the case for microcontrollers **(G1)**. Otherwise, we bypass the kernel policy by manually placing inactive servers on a priority lower than the idle thread and currently active servers on higher priorities, allowing user-level policies to take full control. To speed up the operation, our priority change system call accepts up to two thread capabilities and can update them simultaneously **(G1)**. The trade-off here is to optimize for responsive handling of common scenarios, while accepting an increase in latency for the complex systems that require more *controlled interference* than *absolute minimal latency*.

In *FVM*, the in-kernel FPRR policy is directly in charge of the native processes which provide minimal latency, while the user-level hypervisor scheduler is in charge of the VMs that can tolerate more overhead.

C. Type II Paravirtualizing Hypervisor

The type II hypervisor architecture lies on top of the microkernel to support the guest VMs. The hypercalls are shown in Table I. It performs:

CPU virtualization. Instead of pairing each guest thread with a dedicated microkernel thread, as is commonly done in microkernel-based virtualization, we assign all guest tasks within a VM to a single USR thread, and assign all guest interrupt handlers within a VM to a single VCT thread.

TABLE I
FVM HYPERCALLS. SENSITIVE CONFIGURATIONS CAN BE LOCKED DOWN WITH hyp_vct_lck TO PREVENT FURTHER MODIFICATIONS. THIS GUARANTEES SECURITY AS THE BOOT CODE FOR EACH VM IS STORED IN READ-ONLY FLASH

Hypercall	Functionality
hyp_int_ena	Enable virtual interrupts.
hyp_int_dis	Disable virtual interrupts.
hyp_vct_phys	Register a virtual vector with a physical vector.
hyp_vct_evt	Register a virtual vector with an event source.
hyp_evt_add	Allow this VM to send to an event source.
hyp_vct_lck	**Lockdown vector mappings and event source permissions.**
hyp_vct_wait	Suspend the VM until future vector activation.
hyp_evt_snd	Send an event to an event channel.
hyp_wdg_clr	Start or clear the virtual watchdog.

Compared with the traditional solution, our design (1) eliminates the need to allocate extra microkernel threads **(G2)**, (2) enables user-level guest thread switching without kernel intervention **(G1)**, and (3) avoids the semantic mismatch between the guest RTOS and microkernel scheduler, which eliminates semantic translation and prevents the resulting execution amplification that happen in [7] **(G1)**.

Interrupt indirection. When a physical interrupt arrives, the microkernel unblocks the hypervisor interrupt indirection thread. This thread then searches for registered VMs and injects virtual interrupts into them by unblocking their VCT threads. To use this mechanism, each VM must first call hyp_vct_phys(phys_id, virt_id) to correspond one of its virtual interrupt sources with the physical interrupt. They must then call virt_vct_set(virt_id, vct_ptr) to configure the handler and hyp_vct_lck() to lockdown virtual-to-physical mappings. Finally, they must enable interrupts by calling hyp_int_ena().

In this design, a privileged Dom0 (as in Xen [40]) is not needed since the VM boot code is burned into *read-only* Flash **(G3)**, and each VM is forced to execute it upon booting.

Device management. *FVM* diverges from traditional virtualization by not performing device virtualization. Instead, it passes peripherals through to the VM directly, as they typically do not share peripherals **(G4)**. It is however possible to allocate dedicated native processes or VMs to manage devices, which can then act as a device manager for other VMs **(G3)**. DMA and Ethernet are typical examples: the mismanagement of DMA may breach spatial isolation, while the sharing of Ethernet connectivity provides network access for all VMs.

Event passing. Event sources are similar to physical interrupts in that they also trigger virtual interrupts. They come from VMs and native processes to achieve inter-VM and VM-process communication. Event source setup follows is similar to physical interrupts, and the primary difference is using hyp_vct_evt(evt_id, virt_id) for registration instead. The event mappings are also locked down by hyp_vct_lck(), which prevents further modifications **(G3)**. The VM can then send events to authorized sources through the hyp_evt_snd(evt_id) hypercall.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

VM scheduling. *FVM* employs a Fixed Priority Round-Robin (FPRR) scheduler for scheduling VMs. When a guest VM has no runnable threads, it calls suspends hyp_vct_wait() to suspend its execution until further virtual vector activation, mirroring the behavior of the WFI or WFE instructions in real hardware. When a suspended VM receives a virtual interrupt, it is immediately resumed, and will preempt the current one if it has a higher priority. If a VM does not suspend itself exhausts its allocated budget, the round-robin mechanism will select the next VM for processing. This design deliberately trades interrupt batching efficiency in favor of achieving an absolute minimum latency, thus ensuring real-time responsiveness **(G1)**.

Fault handling. To address faults within VMs, a hypervisor fault handling thread runs at the highest priority. When a fault occurs, this thread unblocks and invokes a customizable handling hook, within which the user can reboot the VM or take other actions **(G3)**. To avoid potential deadlocks or infinite loops from bugs within the VM, *FVM* also provides virtual watchdogs, similar to those found on physical microcontrollers **(G3)**. This is achieved through the hyp_wdg_clr() hypercall, which initializes the watchdog upon the first call and feeds the dog upon subsequent calls.

D. Native Processes

The type-II design enables the coexistence of lighter-weight native processes alongside VMs, similar to KVM-based[2] solutions. The native processes can be used to execute timing-sensitive portions of interrupt "bottom-halves" before passing them to the VM through events.

Process initialization. Typical microcontroller toolchains generate images that are directly executed from persistent on-chip Flash memory. During the boot-up, the initialization code copies the data section (*GCC* .data, *ARMCC* .rwdata) from its Flash load address to the RAM run-time address and clears the zero section (*GCC* .bss, *ARMCC* .zidata).

In certified toolchains such as *ARMCC* and *IAR*, the initialization code and segment details are proprietary. To support them without digging into the obscurity, we masquerade each process as a "bare-metal" image and ensure that no thread can run until the process has been initialized **(G4)**. To achieve this, we identify the highest priority thread within each process and set up the main() function to call it. The key finding is that the highest priority thread must be the first to run within each process, initializing the process before all other code runs.

Latency-sensitive interrupt bottom-halves. In *FVM*, the native processes can receive interrupt activations directly from the microkernel without any indirection. This creates a confined space with less interrupt latency than the VMs, and is ideal for latency-sensitive portions of interrupt "bottom-halves" **(G1)**. The interrupts can be processed in native processes first before triggering events which the hypervisor will translate to virtual interrupt injections. The data associated with the interrupt activations can be passed to VMs through the use of shared memory.

[2]Some sources classify KVM as type-I, but we classify it as type II because it (1) relies on Linux drivers, (2) is separately loadable as a kernel module.

E. Porting RTOSes

The porting of RTOSes involves (1) replacing its tick timer with a virtual one, (2) replacing interrupt enable/disable routines with hypercalls, and (3) porting its thread context switch code.

To refit the tick timer, we register its original timer ISR as a virtual timer interrupt, and configure the hypervisor to periodically inject interrupts. Supplanting interrupt disable/enable is not as straightforward, because replacing them with hyp_int_dis() and hyp_int_ena() introduces hypercalls into all RTOS functions that disable interrupts, causing high overheads. Instead, we provide a pair of library functions, virt_int_mask() and virt_int_unmask(), which mask and unmask the VCT virtual interrupt response without stopping its injection. The virt_int_mask() sets the int_mask flag, which is detected by VCT when it unblocks. The VCT then sets a int_pend flag and ignores all incoming interrupts. When virt_int_unmask() is called, int_mask is cleared, and VCT is signalled to process pending interrupts if the int_pend flag is set. This reduces the interrupt enable/disable overheads to that of the bare-metal **(G1)**.

Interrupt context switch code porting is simple because USR's thread context is exposed to VCT, which allows register manipulation to be written entirely in C instead of assembly, eliminating the most architecture-specific and error-prone aspects of it **(G4)**. This is in stark contrast to Xen [41], where such code must be written in assembly.

F. Project Organization

To seamlessly integrate with the bare-metal ecosystem, we masquerade each system component (the kernel, hypervisor, native processes and VMs) as individual "bare-metal" projects. Compiling each project produces raw binary trunks (.bin or .hex) that are transcribed into data arrays later. This strips away all potentially conflicting library symbols without requiring executable interlinkability between projects. The arrays are then mapped to fixed addresses as raw data in the microkernel project to create a combined system image. These steps make no assumptions beyond the core functionality of the toolchains, ensuring compatibility across all of them **(G4)**.

IV. *FVM* Implementation

To demonstrate the general applicability of *FVM*, we carry out implementations across four popular chips of three common architectures, two industry-standard toolchains, and two open-source RTOSes, for a total of seven combinations shown in Table II.

A. FVM for ARMv6-M: The Minimal

At the lower end, we choose the *STM32L071CBT6* featuring 128KiB Flash and 20KiB RAM, paired with a MPU-enabled *ARMv6-M Cortex-M0+* core @ 32MHz.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

TABLE II
FVM IMPLEMENTATION COMBINATIONS

Chip	CPU	Flash/RAM	Toolchain	RTOS
STM32L071CBT6	CM0+	128K/20K	ARMCC/GCC	FreeRTOS/RMP
STM32F405RGT6	CM4F	512K/192K	ARMCC/GCC	FreeRTOS/RMP
STM32F767IGT6	CM7F	1M/512K	ARMCC/GCC	FreeRTOS/RMP
CH32V307VCT6	RV32	192K/128K	GCC	FreeRTOS/RMP

Listing 1. Assembly.

```
PendSV_Handler PROC
MRS     R0,  PSP
LDR     R2,  =pxCurrentTCB
STR     R0,  [R2]
STMIA   R0!, {R4-R7}
MOV     R4,  R8
...
STMIA   R0!, {R4-R7}

PUSH    {R3, LR}
BL      vTaskSwitchContext
POP     {R2, R3}

LDR     R0,  [R2]
MOV     R1,  R0
ADDS    R0,  R0, #16
LDMIA   R0!, {R4-R7}
MOV     R8,  R4
...
LDMIA   R1!, {R4-R7}
MSR     PSP, R0
BX      R3
ENDP
```

Listing 2. Transcribed C.

```
void VCT_PendSV_Handler(void) {
    unsigned long* SP;
    SP = USR_REG->Reg.SP;

    *(--SP) = USR_REG->Reg.R4;
    ....
    *(--SP) = USR_REG->Reg.R11;
    *(--SP) = USR_REG->Reg.LR;

    *pxCurrentTCB = SP;
    vTaskSwitchContext();
    SP = *pxCurrentTCB;

    USR_REG->Reg.LR = *(SP++);
    USR_REG->Reg.R11 = *(SP++);
    USR_REG->Reg.R10 = *(SP++);
    USR_REG->Reg.R9 = *(SP++);
    USR_REG->Reg.R8 = *(SP++);
    ....
    USR_REG->Reg.R4 = *(SP++);
    USR_REG->Reg.SP = SP;
    return;
}
```

Memory management. The *Cortex-M0+* MPU has 8 regions with power-of-2 sizes, where each region's base address must align with its size. The system composer's static region allocation algorithm is similar to that of [7], which tries to minimize the number of regions required for a given memory map by considering both regions and subregions.

The *Cortex-M0+* faults upon memory access permission violations, but does not provide address information. This makes the dynamic region swapping algorithm in [42] inapplicable, and there's little additional point to keep the in-kernel memory management facility. Thus, we disable it and store generated MPU protection tables in read-only Flash.

RTOS porting. We port two open-source RTOSes, *FreeRTOS* and *RMP*, to the *FVM*. We registered their original timer interrupts to the virtual timer source using virt_vct_set(), and replaced their interrupt disable/enable calls with virt_int_mask() and virt_int_unmask(), respectively. We also replaced system initialization code with a simple hyp_int_ena() call. No further change beyond the context switch code is required.

Porting *FreeRTOS* context switch assembly is straightforward as it relies solely on the PendSV_Handler() software interrupt triggered by the portYield(). We translate the assembly vector (Listing 1) to C (Listing 2), and register it with the VCT. The resulting C code is simpler than the assembly due to *Cortex-M0+*'s not supporting LDMIA on high registers R7-R11.

The *RMP* context switch code additionally included an assembly snippet that switches context without triggering the PendSV interrupt. To integrate it, we replaced its CPSID and CPSIE with virt_int_mask() and virt_int_unmask() calls, respectively. We fitted this snippet to the *FreeRTOS* as well to improve its performance.

Sensitive register fixes. We notice that the LR stores information about where an interrupt would return, with 0xFFFFFFF1 indicating HANDLER mode (kernel-level) and 0xFFFFFFFD indicating THREAD mode (user-level). Hence, malicious LR modifications by VCT can lead to VM execution with kernel privileges, and we defeat this by forcing LR to 0xFFFFFFFD upon interrupt exits.

B. FVM for ARMv7-M: The Mainstream

The *ARMv7-M* (*Cortex-M4F* and *Cortex-M7F*) adds optional FPU and DSP instructions to the base *ARMv6-M* instruction set. Compared with the prevalent *Cortex-M4F*, the rarer *Cortex-M7F* has a higher clock rate, better IPC, and extra double-precision floating point capabilities. We selected the 168MHz *STM32F405RGT6* that has 512KiB Flash and 192KiB RAM to represent the *Cortex-M4F*, and the 216MHz *STM32F767IGT6* 1MiB Flash and 512KiB RAM to represent the *Cortex-M7F*. Though *ARMv7-M* chips are significantly faster than *ARMv6-M* ones, we don't detail the basic porting as it remains similar. We focus on FPU management instead, which is unique for *ARMv7-M*.

FPU management. The *Cortex-M4F* and *Cortex-M7F* has identical FPU registers despite that the latter support double-precision. Their FPU context switching is hence the same from a system standpoint.

The FPU has two unique features: automatic stacking (FPCCR.ASPEN) which pushes FPU registers to the user-level stack upon interrupts if FPU is DIRTY, and lazy stacking (FPCCR.LSPEN) which postpones such pushing until FPU is accessed during a context switch. The former minimizes interrupt latency, while the latter avoids excessive pushes when context switches are absent. For optimal performance, we enable both features, and the lazy stacking may occur when the microkernel saves the FPU context into the thread kernel object. A malicious thread may set its stack pointer to invalid addresses to trigger faults in kernel-space. To this end, we ignore lazy stacking errors (LSPERR and MLSPERR); any malicious FPU stacking attempts only harm the attackers.

C. FVM for RISC-V: The Emerging

The *RISC-V* is an emerging architecture that targets scenarios spanning from cloud computing to embedded. Its openness makes it suitable for microcontroller designs that require flexibility. We choose the 144MHz *CH32V307VCT6* (*RV32IMAFC* core) that has 192KiB Flash and 128KiB RAM to represent *RISC-V*. This port proved the most challenging and we elaborate on the unique difficulties below.

Nonstandard hardware caveats. The *RISC-V* is an open architecture and manufacturers are *free* to implement their own flavor despite the ratified standards. Unfortunately, this flexibility is not without downsides. Take the *CH32V307VCT6* as an example, it (1) is missing the mandatory machine mode timer

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

(mtime), (2) does not generate precise fault addresses, (3) has a modified interrupt vector table, (4) and so on. We employ custom code and workflow changes for these nonstandard hardware to make the port possible.

Memory region allocation. The *RISC-V* PMP is unique in that it supports both Naturally-Aligned-Power-Of-Two (NAPOT) and Top-Of-Range (TOR) modes. In NAPOT mode, a single entry defines a power-of-2 aligned memory segment similar to the *Cortex-M* MPU. The TOR mode however allows declaring unaligned segments using two entries, each specifying the base and limit address. To use the regions efficiently, our algorithm merges contiguous segments, and checks if the merged segment can fit into a NAPOT entry; if not, we use two entries working in TOR mode instead. Due to the absence of precise PMP faults, the dynamic region swapping algorithm [42] cannot be used.

V. EVALUATION

In our evaluation, we examine the following key questions that directly correspond to our goals listed in Section III:

Q1: How much processing overhead does the *FVM* introduce and is this acceptable?

Q2: How much memory bloat does the *FVM* impose and is this acceptable?

Q3: What changes to a RTOS is needed to port it to the *FVM* and is this acceptable?

Q4: Can the *FVM* recover from VM faults or attacks and reboot the affected VMs individually?

To address these questions, we perform a comprehensive benchmark on four chips introduced in Section IV. In all benchmarks, we run two native processes alongside one VM. We use *GCC* 13.2.1 and *ARMCC* 6.3.1 for compilation with the -O3 compiler flag, as well as *FreeRTOS* 9.0.0 and *RMP* 0.2.7 for virtualization evaluation. The *Cortex-M0+* uses an -Os size optimization flag instead, which better reflects the real-world scenarios.

Each benchmark is repeated 10000 times and we report both the mean (average; darker bottom bar) and worst-case (maximum; lighter top bar) values. We do not provide standard deviation values as they are virtually invisible in graphs.

A. Bare-Metal Performance (*Q1*)

We start by benchmarking the baseline performance of vanilla *FreeRTOS* and *RMP* using both *GCC* and *ARMCC* toolchains, which is shown in Figs. 3 and 4. *ARMCC* is only compatible with the ARM architecture and thus does not apply to the *RISC-V* processor.

For *FreeRTOS*, we measure the (1) context switch latency (`ctx`), (2) notification latency (`ntf`), (3) semaphore latency (`sem`), (4) message-queue latency `msg`, and (5) their respective interrupt versions with (`.../i`).

For *RMP*, we measure the (1) context switch latency (`ctx`), (2) mailbox latency (`mbx`), (3) semaphore latency (`sem`), (4) message-queue latency `msg`, (5) bounded message-queue latency `bmsg` and (6) their respective interrupt versions (`.../i`).

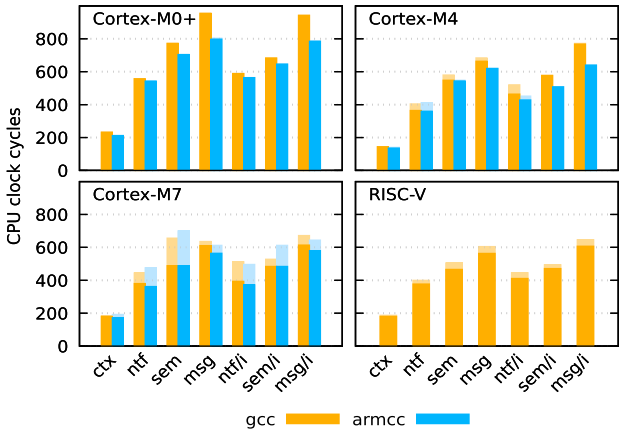

Fig. 3. Bare-metal *FreeRTOS* performance. "`ctx`" is one-way context switch overhead (taskYIELD), "`ntf`" is notification latency (xTaskNotify), "`sem`" is semaphore latency (xSemaphoreGive), "`msg`" is message-queue latency (xQueueSend), and "`.../i`" are respective interrupt latencies (...FromISR). The bare-metal performance typically falls within several microseconds, except for the *Cortex-M0+* which has a low clock rate.

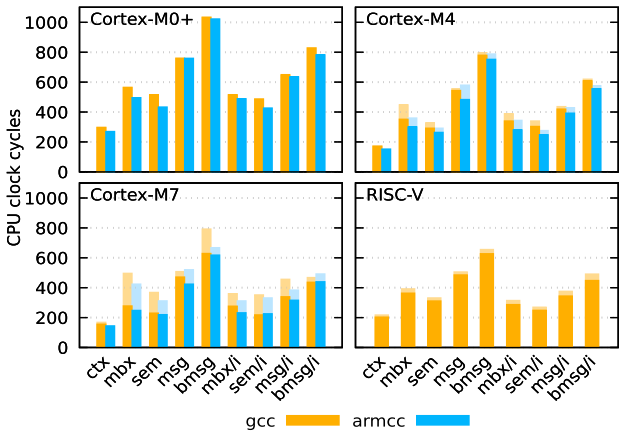

Fig. 4. Bare-metal *RMP* performance. "`ctx`" is one-way context switch overhead (Thd_Yield), "`mbx`" is thread mailbox latency (Thd_Snd), "`sem`" is semaphore latency (Sem_Post), "`msg`" is message-queue latency (Msgq_Snd), "`bmsg`" is bounded-size message-queue latency (Bmq_Snd) and "`.../i`" are respective interrupt latencies (..._ISR). Notably, the *RMP*'s semaphore ("`sem`") is faster than *FreeRTOS*'s, but bounded message queue ("`bmsg`") is slower than *FreeRTOS*'s; this is due to *RMP*'s semaphore-centric construction as opposed to the *FreeRTOS*'s queue-centric construction.

RMP's mailbox is roughly equivalent to the *FreeRTOS*'s notification as it is tied to a thread and allows to pass light-weight messages between them. *RMP*'s message queues are separated into regular unbounded queues and bounded queues, and the latter is equivalent to *FreeRTOS* message queues.

Discussion. On all architectures and toolchains, both RTOSes perform well and almost all system operation latencies stayed below 1000 CPU cycles. This translates to a few microseconds as all processors but the *Cortex-M0+* runs at more than 100 MHz. As these results are what is achievable by vanilla RTOSes, they represent overheads that are acceptable in applications that already leverage these RTOSes.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

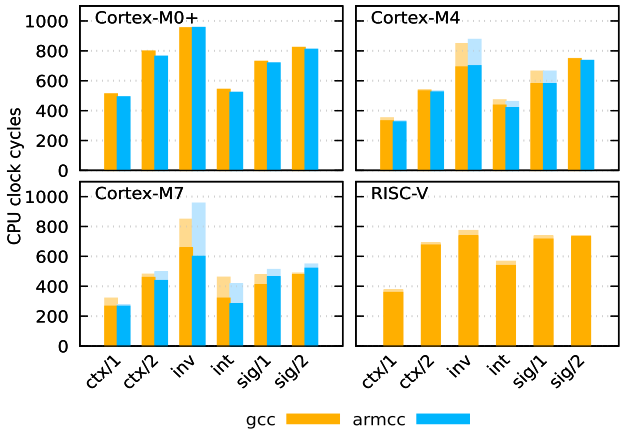

Fig. 5. Microkernel basic operation performance. It is somewhat comparable to that of both RTOSes, although it exhibits slightly higher latencies due to the need to reprogram the MPU during protection domain switches.

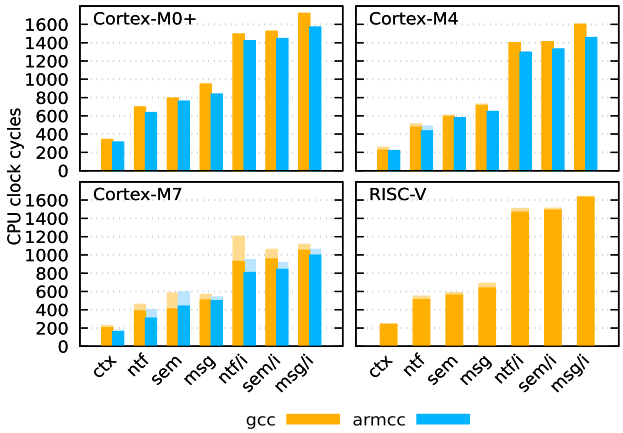

Fig. 6. Virtualized *FreeRTOS* performance. The abbreviations in here and Fig. 8 have the same meaning as in Fig. 3.

B. Microkernel Performance (*Q1*)

As the *FVM* relies on the underlying microkernel, the microkernel performance must be benchmarked so that we can understand other evaluations that follow. As shown in Fig. 5, we measure (1) intra-process context switch latency (`ctx/1`), (2) inter-process context switch latency (`ctx/2`), (3) thread migration/return round-trip latency (`inv`), (4) interrupt signaling latency (`int`), (5) intra-process signaling latency (`sig/1`), and (6) inter-process signaling latency (`sig/2`).

Discussion. The *FVM*'s underlying microkernel is comparable to the bare-metal RTOSes in terms of intra-process operations despite its addition of capability-based security. However, in inter-process measurements, an additional 100 cycle overhead is imposed by MPU reprogramming operations, and this translates to less than one microsecond on all processors but the *Cortex-M0+*. The microkernel has less interrupt latency than both RTOSes, because its signal endpoint is lighter-weight than RTOS message-passing IPCs. A signal delivers just the activations without the message, while the messages can be passed in shared memory when needed. Thread migration is also provided for synchronous communication, which is twice as fast as an asynchronous signal round-trip.

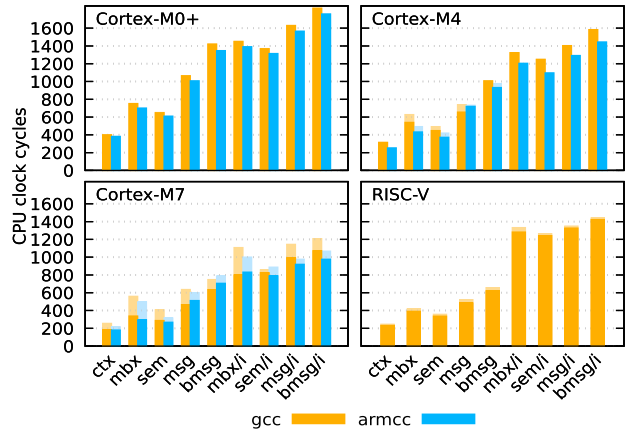

Fig. 7. Virtualized *RMP* performance. The abbreviations in here and Fig. 9 have the same meaning as in Fig. 4.

C. Guest Virtualization Performance (*Q1*)

To evaluate how the RTOSes perform after they have been virtualized, we measure the same system operation latencies as we've measured in bare-metal environment, which is shown in Figs. 6 and 7.

Discussion. We focus on two groups of latencies. First, the context switch and IPC latencies between the threads. Second, the interrupt latencies from the virtual interrupt handler to the receiving thread. Our results show that for both RTOSes, the first group of latencies remains similar to bare-metal performance. This is because a single USR thread corresponds to all guest threads, and guest thread switching can occur at the user-level without invoking the microkernel. However, when it comes to the second group where IPCs originating from virtual interrupts, the results are different. Even when we expose the USR context to the VCT so that it can be modified efficiently without additional system calls, the resulting latencies remain much higher than those of bare-metal. This is due to the fact that the virtual interrupt injection causes two kernel-level context switches: from USR to VCT upon virtual interrupt activation, and from VCT back to USR upon virtual interrupt exiting. Note that these context switches are unavoidable even if we map one guest thread to one microkernel thread.

When we normalize virtualized measurements by bare-metal measurements as shown in Figs. 8 and 9, these observations become much more pronounced. The performance bloat of the first group remained below 60% for all combinations of toolchains and architectures. In a few measurements involving more substantial guest system operations, the difference is hardly noticeable, *i.e. FreeRTOS*'s IPC virtualization overhead is below 5% in *Cortex-M7F*. The second group shows a significant performance penalty: the virtual interrupt latencies overheads are at least 180% of their bare-metal counterpart and can reach up to 500% in some cases, *i.e. RMP*'s semaphore interrupt latency in *RISC-V*.

From these results we can safely conclude that keeping timing-sensitive portions of interrupt "bottom-halves" within

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

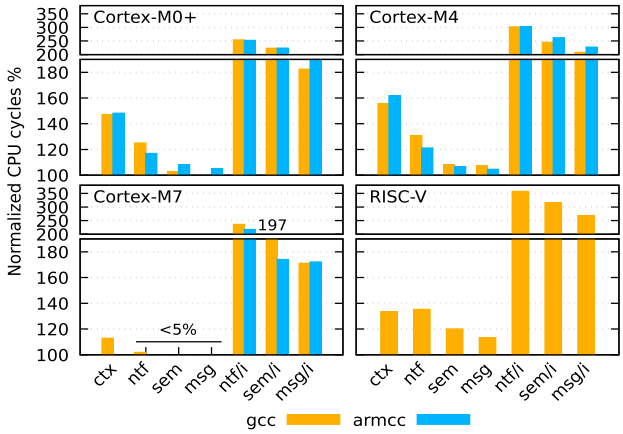

Fig. 8. Virtualized *FreeRTOS* performance normalized by the original bare-metal performance that serves as the 100% baseline.

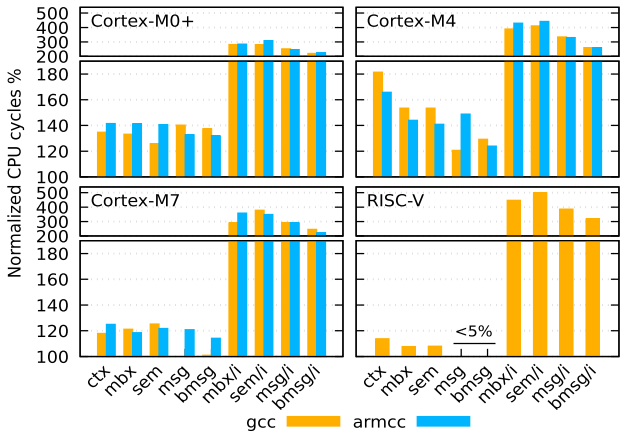

Fig. 9. Virtualized *RMP* performance normalized by the original bare-metal performance that serves as the 100% baseline.

guest threads is a disaster regardless of how optimized the virtualization infrastructure might be.

D. Interrupt Latency (*Q1*)

To evaluate the real-time responsiveness of the virtualized RTOSes, we investigate the latency overhead imposed by *FVM*. Instead of showing the raw numbers, we normalize them by various bare-metal RTOS interrupt latencies to make a cleaner comparison, which is shown in Figs. 10 and 11. We also show the native process interrupt latency side-by-side, which is normalized as well.

Discussion. The interrupt relaying latency is generally 400% of any bare-metal latency, and can reach as high as 800% in some cases (`v/...`). This underpins the fact that even placing latency-sensitive portions of interrupt "bottom-halves" into virtual interrupt handlers, where the bare-metal "top-halves" originally reside, is not a viable solution.

However, our type-II hypervisor design shines here: we can relocate latency-sensitive portions of the "bottom-halves" into native processes, where they enjoy native process interrupt latency which is generally on par with that of the bare-metal.

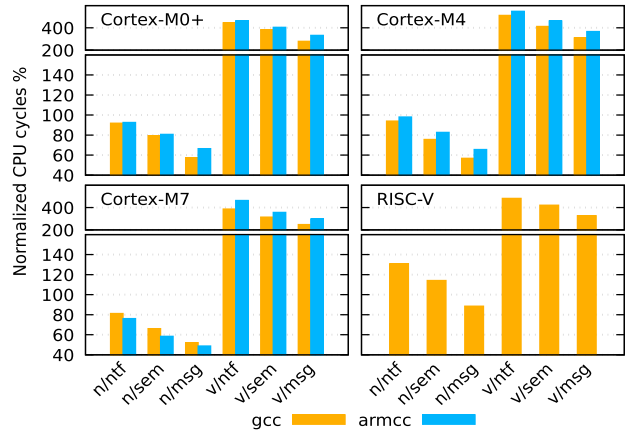

Fig. 10. Native process and VM interrupt relay latency normalized by various bare-metal *FreeRTOS* interrupt latencies that serve as the 100% baseline. The "`n/...`" normalize native process interrupt latency to various bare-metal *FreeRTOS* interrupt latencies, while the "`v/...`" normalize VM interrupt relay latency to various *FreeRTOS* bare-metal interrupt latencies. The abbreviations here have the same meaning as in Fig. 3.

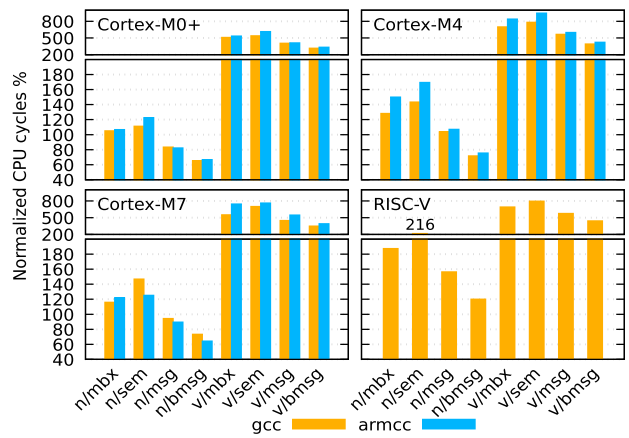

Fig. 11. Native process and VM interrupt relay latency normalized by various bare-metal *RMP* interrupt latencies that serve as the 100% baseline. The notations and abbreviations here have the same meaning as in Figs. 4 and 10.

Due to the fact that the microkernel signals only contain activations but no messages, the pure interrupt latency is actually lower than that of the bare-metal RTOS which carries messages, and could reach as low as 50% *i.e.* in the case of *FreeRTOS* on *Cortex-M7F*. In a word, when the "bottom-halves" are moved to native processes, the *FVM* has comparable interrupt latency to bare-metal systems.

Combining Sections V-A, V-C, V-D and specific application requirements in [43], we can safely conclude that the virtualization latencies are acceptable in most applications that already adopt bare-metal RTOSes.

E. Power Draw (*Q1*)

To perceive the processing overheads from a power draw perspective, we measure the microcontroller (with minimum external circuitry) 3.3V supply currents when running the benchmark and report the average, as shown in Table III.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

TABLE III
AVERAGE CURRENT (MA) WHEN RUNNING THE BENCHMARK NONSTOP. THE "v..." ENTRIES REPRESENT THE VIRTUALIZED RTOSES

Toolchain	Test	*Cortex-M0+*	*Cortex-M4F*	*Cortex-M7F*	*RISC-V*
GCC	*FreeRTOS*	7.85	53.10	90.13	31.98
	v*FreeRTOS*	9.41	60.66	107.44	32.60
	RMP	8.31	50.90	94.94	32.36
	v*RMP*	9.51	55.62	107.12	33.04
ARMCC	*FreeRTOS*	7.93	53.69	91.03	N/A
	v*FreeRTOS*	9.48	60.07	105.45	N/A
	RMP	8.26	50.03	96.64	N/A
	v*RMP*	9.43	56.20	106.35	N/A

Discussion. It could be observed that the power overheads are mild in all cases – some 10% increase – which can be attributed to extra CPU mode switches and enabling of MPUs. However, this difference is only exposed when the system is exercised, and real-world applications rarely exercise system calls as strong as benchmarks do. Even in cases where they do, a 10% increase is quite acceptable when compared to SFI approaches that can cause 100% overheads on all execution, including that of the application [31].

F. Comparison Study (*Q1*)

To better understand the contributions of this work, we compare it to the existing *Composite* virtualization framework [7] which only requires standard MPUs. To make an apples-to-apples comparison, we use the GCC version 5.4.1, and reduce optimization to -O2 to match [7].

The normalized *FVM* performance results are presented in Fig. 12, where the *Composite* values serve as the baseline.

The microkernel inter-process context switch latency (`kctx/2`), thread migration latency (`kinv`), inter-process asynchronous signal latency (`ksig/2`) and interrupt latency (`kint`) are normalized by their corresponding *Composite* kernel values. All these values are inter-process measurements and are shown by the solid bars. Additionally, the *FVM* virtualized *FreeRTOS* context switch latency (`fctx`), message queue latency (`fmsg`) and message queue interrupt latency (`fmsg/i`) are normalized by corresponding *Composite* virtualized *FreeRTOS* values. These are shown by the striped bars. To demonstrate the effectiveness of user-level context switches, we disable the fast-path in our virtualized *FreeRTOS* and perform all context switches through VCT activations. The context switch latency (`fctx/s`) and message queue latency (`fmsg/s`) without user-level fast-path are normalized by *Composite* virtualized *FreeRTOS* values as well.

Discussion. *FVM* microkernel outperforms *Composite* microkernel in most operations (except for `kinv`, which *Composite* heavily optimizes) thanks to its simplified scheduling facilities. Among all measurements, the interrupt latency is significantly reduced by approximately 60% (`kint`), which is more than twice as fast. Despite *Composite*'s *TCaps* mechanism allowing for direct VM interrupt delivery without hypervisor indirection, our design's native process interrupt latency remains superior.

Virtualized *FreeRTOS* measurements demonstrate that our virtualized *FreeRTOS* context switch is over five times faster than that of *Composite* (`fctx`). While message queue latencies are also vastly improved (`fmsg`), it is somewhat less pronounced due to the overheads associated with message passing. This performance boost is attributed to user-level guest context switches that don't require microkernel intervention, as well as the decoupling of guest context switches and microkernel context switches.

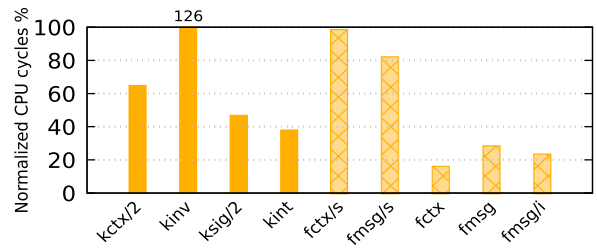

Fig. 12. Microkernel and virtualized *FreeRTOS* performance normalized by that of [7], which serves as the 100% baseline. The solid bars ("`k...`") represent microkernel measurements, while the striped bars ("`f...`") correspond to virtualized *FreeRTOS* measurements. The abbreviations here have the same meaning as in Figs. 3 and 5; the additional "`.../s`" do not use the fast context switching path.

The four fold improvement in message queue interrupt latency (`fmsg/i`) can also be attributed to simplified scheduling facilities. This reason is somewhat intricate: *Composite* corresponds each *FreeRTOS* thread to a *Composite* thread and thus needs to translate *FreeRTOS* scheduling semantics to *Composite TCaps* scheduling semantics. In contrast, our system doesn't require this translation as we model the vCPU instead of making a direct correspondence.

Even with the *Composite*'s *Slite* user-level context switching extension enabled [44], *Composite* is still slower than *FVM*. Take context switch as an example, *Composite-Slite* virtualized *FreeRTOS* has a latency of 2071 cycles on *Cortex-M33F* [44], whereas *FVM* virtualized *FreeRTOS* only have a latency of 224 cycles on *Cortex-M4F*, despite *Cortex-M33F* being about 15% more performant IPC-wise. This is due to the *Composite*'s many-to-many threading model, as well as its additional cost of synchronizing user-level scheduler states with the kernel.

In a word, a straightforward FPRR scheduler is better suited for microcontrollers with simple microarchitectures than MCS-capable scheduling facilities; the latter would cause unacceptable overhead.

G. Memory Footprint (*Q2*)

To understand how scalable the system is on limited microcontroller on-chip memory, we compile each system component and delineate their memory usage, as shown in Table IV. We also provide bare-metal numbers including and excluding drivers for comparison. Note that the numbers already include all run-time created kernel objects, and both RTOSes are built with a comprehensive benchmark that thoroughly exercises their system calls, allowing for a practical memory footprint to be reflected.

Discussion. The microkernel always uses the most Flash due to its complexity. The hypervisor is a close second as it contains the bulk of VM scheduling, event passing, interrupt relaying,

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
MEMORY FOOTPRINTS OF ALL SOFTWARE COMPONENTS, IN KILOBYTES. THE NUMBERS ARE LISTED IN A FLASH/RAM FORMAT, WITH THE FIRST REPRESENTING FLASH AND THE SECOND REPRESENTING RAM. THE "`d...`" ENTRIES DENOTE THE BARE-METAL VERSIONS OF THE TWO RTOSES WITH MINIMAL NECESSARY DRIVERS, THE "`b...`" ENTRIES DENOTE THEIR BARE-METAL VERSIONS WITH ALL DRIVERS STRIPPED, AND THE "`v...`" ENTRIES REPRESENT THEIR VIRTUALIZED COUNTERPARTS

Toolchain	Component	*Cortex-M0+*	*Cortex-M4F*	*Cortex-M7F*	*RISC-V*
GCC	Kernel	55.8/5.89	72.0/6.72	72.6/8.83	65.5/6.33
	Hypervisor	15.9/1.86	21.5/2.28	22.0/2.50	22.2/2.45
	d*FreeRTOS*	14.7/2.61	17.8/5.22	17.0/4.91	23.5/4.91
	b*FreeRTOS*	9.8/2.60	14.4/4.85	11.8/4.90	17.9/4.85
	v*FreeRTOS*	10.8/2.47	14.1/4.70	14.3/4.70	14.7/4.71
	d*RMP*	16.1/1.67	21.0/1.70	24.9/1.98	27.9/1.70
	b*RMP*	11.4/1.66	19.5/1.68	19.6/1.96	25.2/1.68
	v*RMP*	13.2/1.55	20.4/1.56	20.6/1.56	25.8/1.53
ARMCC	Kernel	55.5/5.89	95.0/6.72	94.7/8.83	N/A
	Hypervisor	15.0/1.86	41.3/2.28	31.5/2.50	N/A
	d*FreeRTOS*	12.1/2.58	14.2/4.98	14.6/4.88	N/A
	b*FreeRTOS*	7.8/2.57	11.3/5.11	10.5/4.87	N/A
	v*FreeRTOS*	9.1/2.46	12.8/4.69	13.0/4.69	N/A
	d*RMP*	13.4/1.64	21.8/1.65	22.0/1.95	N/A
	b*RMP*	9.3/1.63	19.7/1.64	17.8/1.93	N/A
	v*RMP*	11.1/1.53	20.0/1.56	20.0/1.56	N/A

hypercall handling, and fault handling. However, we argue that Flash is inexpensive and abundant on microcontrollers, as a sub-$1 *STM32G0B1RET6* has 512K Flash.

The microkernel and hypervisor uses approximately 10K RAM, and can be easily fit into microcontrollers that have more than 64K RAM. When the microkernel is used without the hypervisor, the RAM footprint shrinks to about 3K, which could fit into any microcontroller with 16K or more RAM.

The overheads for RTOSes are quite interesting. When the footprints without any drivers (`b...`) are compared to their virtualized counterparts (`v...`), the virtualized versions use more Flash/RAM due to their inclusion of the paravirtualizing library. However, when looking at the footprints with common manufacturer-provided drivers *e.g. STM HAL* (`d...`), the virtualized versions are less bloated because they do not contain hardware drivers anymore. This means that a consolidated system actually use *less* memory than the original system combined, which reflects reduced costs.

Thanks to our simpler kernel design, when the footprints are compared to [7], we decrease microkernel Flash and RAM usage by 46% and 76%, and VM Flash and RAM usage by 72% and 84%, respectively. The removal of unnecessary kernel object alignment requirements is also a contributing factor. In conclusion, the *FVM* memory footprints are acceptable on commodity microcontrollers.

H. RTOS Porting Effort (*Q3*)

The *FVM* is a paravirtualization facility that requires modifications to the guest OS or firmware. To assess the adaptation effort on existing microcontroller code, we measure the Source Lines of Code (SLoC) that must be modified to port both RTOSes, as shown in Table V.

Discussion. The modifications to the C source code are relatively minor, with around 50 lines changed for both *ARMv6-M* (*Cortex-M0+*) and *ARMv7-M* (*Cortex-M4F* / *Cortex-M7F*), and 120 lines changed for *RISC-V*. Compared to the SLoC of the bare-metal RTOS hardware abstraction layers (in the case of *ARMv7-M FreeRTOS*, 483 lines), these modifications only accounted for only about 1/5 of the original code.

TABLE V
MODIFICATIONS OF RTOS SOURCE, IN SOURCE LINES OF CODE (SLOC). THE NUMBERS ARE LISTED IN A +/- "DIFF" FORMAT, INDICATING ADDITIONS (+) AND DELETIONS (-). THE FASTYIELD ENTRIES DENOTE THE ADDITIONAL USER-LEVEL ASSEMBLY CODE THAT ACCELERATES GUEST THREAD CONTEXT SWITCHES

RTOS	File	*ARMv6-M*	*ARMv7-M*	*RISC-V*
FreeRTOS	C source	+51/-23	+48/-33	+116/-38
	ARMCC asm	+22/-49	+16/-46	N/A
	+fastyield	+111	+91	N/A
	GCC asm	+22/-49	+16/-46	+8/-114
	+fastyield	+108	+87	+122
RMP	C source	+49/-3	+50/-4	+101/-5
	ARMCC asm	+1/-45	+1/-33	N/A
	+fastyield	+109	+91	N/A
	GCC asm	+1/-45	+1/-33	-115
	+fastyield	+106	+91	+122

We also compare the total SLoC in the virtualized *FreeRTOS* HAL to [7], where our *FreeRTOS* HAL includes only 283 SLoC instead of [7]'s 363 SLoC. This is because the *FVM* don't need to translate *FreeRTOS* scheduling semantics to the microkernel's.

I. VM Anomaly Detection and Recovery (*Q4*)

To evaluate the system's resilience to errors or attacks, we intentionally inject references to (1) null pointers, (2) memories of other protection domains, and (3) nonexistent capabilities (4) into the otherwise correct VM logic.

Discussion. For all combinations of toolchains, architectures, and RTOSes, *FVM* can detect the anomaly immediately and reboot the offending VM without affecting the entire microcontroller, which accomplishes its design goals. However, *FVM* does not defend against ROPs which does not cause permission violations, and this can be complemented by CFI techniques such as sandboxes [45], [31]. In is also noteworthy that sandboxes can also be used in lieu of *FVM* to achieve SFI on even legacy microcontrollers that does not feature a MPU, at the cost of much higher execution overheads [31].

VI. CONCLUSION

The need to reduce costs, improve reliability and enhance security has been driving microcontroller consolidation in automotive, industrial and commercial scenarios. To this end, the paper proposes *FVM*, a virtualization framework for performing practical consolidation on commodity microcontrollers. We optimize the *FVM* design towards the microcontroller ecosystem and implement it on four chip architectures, three industry-standard toolchains, two open-source RTOSes, for a total of seven combinations. As the evaluation demonstrates, *FVM* reduces the latency overheads down to *near zero*, reduces the virtualization footprint bloat by up to 72%, and imposes minimal effort to port existing systems to it. We believe that *FVM* enables practical virtualization-based microcontroller consolidation in automotive, industrial, and commercial applications.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] K. Vipin, "CANNoC: An open-source NoC architecture for ECU consolidation," in *Proc. IEEE Int. Midwest Symp. Circuits Syst. (MWSCAS)*, 2018.

[2] "Adaptive AutoSAR. The AUTOSAR runtime for adaptive applications (ARA)." AutoSAR. Accessed: Dec. 14, 2020. [Online]. Available: https://www.autosar.org/standards/adaptive-platform

[3] F. Paci, D. Brunelli, and L. Benini, "Lightweight IO virtualization on MPU enabled microcontrollers," in *Proc. Embedded Oper. Syst. Workshop Co-Located Embedded Syst. Week (ESWEEK)*, 2016.

[4] C. Morales-Gonzalez et al., "On building automation system security," *High-Confidence Comput.*, vol. 4, no. 3, p. 100236, 2024.

[5] B. Sá, J. Martins, and S. Pinto, "A first look at RISC-V virtualization from an embedded systems perspective," *IEEE Trans. Comput.*, vol. 71, no. 9, pp. 2177–2190, Sep. 2022.

[6] Z. Jiang, K. Yang, Y. Ma, N. Fisher, N. Audsley, and Z. Dong, "Towards hard real-time and energy-efficient virtualization for many-core embedded systems," *IEEE Trans. Comput.*, vol. 72, no. 1, pp. 111–126, Jan. 2023.

[7] R. Pan, G. Peach, Y. Ren, and G. Parmer, "Predictable virtualization on memory protection unit-based microcontrollers," in *Proc. 24th IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2018, pp. 62–74.

[8] N. Klingensmith and S. Banerjee, "Hermes: A real time hypervisor for mobile and IoT systems," in *Proc. 19th Int. Workshop Mobile Comput. Syst. Appl.*, 2018, pp. 101–106.

[9] S. Pinto, H. Araujo, D. Oliveira, J. Martins, and A. Tavares, "Virtualization on TrustZone-enabled microcontrollers? Voilà!" in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2019, pp. 293–304.

[10] A. K. Sundar Rajan, A. Feucht, L. Gamer, I. Smaili, and M. Nirmala Devi, "Hypervisor for consolidating real-time automotive control units: Its procedure, implications and hidden pitfalls," *J. Syst. Archit.*, vol. 82, pp. 37–48, 2018.

[11] FreeRTOS. "Real-time operating system for microcontrollers and small microprocessors," Accessed: May 1, 2013. [Online]. Available: http://www.freertos.org

[12] W. Zhou, L. Guan, P. Liu, and Y. Zhang, "Good motive but bad design: Why ARM MPU has become an outcast in embedded systems," 2019, *arXiv:1908.03638*.

[13] D. Danner, R. Muller, W. Schröder-Preikschat, W. Hofer, and D. Lohmann, "SAFER SLOTH: Efficient, hardware-tailored memory protection," in *Proc. 20th IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2014, pp. 37–48.

[14] L. Amit et al., "Multiprogramming a 64 kB computer safely and efficiently," in *Proc. 26th Symp. Oper. Syst. Princ. (SOSP)*, 2017, pp. 234–251.

[15] A. A. Clements et al., "Protecting bare-metal embedded systems with privilege overlays," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2017, pp. 289–303.

[16] A. A. Clements, N. S. Almakhdhub, S. Bagchi, and M. Payer, "ACES: Automatic compartments for embedded systems," in *Proc. 27th USENIX Conf. Secur. Symp. (CSS)*, 2018, pp. 65–82.

[17] D. Kwon, J. Shin, G. Kim, B. Lee, Y. Cho, and Y. Paek, "uXOM: Efficient execute-only memory on ARM Cortex-M," in *Proc. USENIX Secur. Symp. (USENIX Secur.)*, 2019, pp. 231–247.

[18] N. Dejon, C. Gaber, and G. Grimaud, "Pip-MPU: Formal verification of an MPU-based separation kernel for constrained devices," *Int. J. Embedded Syst. Appl.*, vol. 13, no. 2, pp. 1–21, 2023.

[19] "Mbed µVisor." Accessed: Feb. 6, 2024. [Online]. Available: https://github.com/armmbed/uvisor

[20] Z. B. Aweke and T. Austin, "uSFI: Ultra-lightweight software fault isolation for IoT-class devices," in *Proc. Des., Automat. Test Europe Conf. Exhib. (DATE)*, 2018, pp. 1015–1020.

[21] X. Zhou, J. Li, W. Zhang, Y. Zhou, W. Shen, and K. Ren, "OPEC: Operation-based security isolation for bare-metal embedded systems," in *Proc. 17th Eur. Conf. Comput. Syst. (EuroSys)*, 2022, pp. 317–333.

[22] "F9 microkernel." GitHub. Accessed: Feb. 6, 2024. [Online]. Available: https://github.com/f9micro/f9-kernel

[23] L. Zhao, G. Li, B. De Sutter, and J. Regehr, "ARMor: Fully verified software fault isolation," in *Proc. 9th ACM Int. Conf. Embedded Softw. (EMSOFT)*, 2011, pp. 289–298.

[24] A. Khan, D. Xu, and D. J. Tian, "Low-cost privilege separation with compile time compartmentalization for embedded systems," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 2023, pp. 3008–3025.

[25] C. H. Kim et al., "Securing real-time microcontroller systems through customized memory view switching," in *NDSS*, 2018.

[26] P. Koeberl, S. Schulz, A.-R. Sadeghi, and V. Varadharajan, "TrustLite: A security architecture for tiny embedded devices," in *Proc. 9th Eur. Conf. Comput. Syst. (EuroSys)*, 2014.

[27] K. Zandberg, E. Baccelli, S. Yuan, F. Besson, and J.-P. Talpin, "Femto-containers: Lightweight virtualization and fault isolation for small software functions on low-power IoT microcontrollers," in *Proc. ACM/IFIP Int. Middleware Conf. (Middleware)*, 2022, pp. 161–173.

[28] "MicroPython for microcontrollers." MicroPython. Accessed: Jun. 10, 2017. [Online]. Available: http://www.micropython.org

[29] R. J. Walls, N. F. Brown, T. L. Baron, C. A. Shue, H. Okhravi, and B. C. Ward, "Control-flow integrity for real-time embedded systems," in *Proc. 31st Euromicro Conf. Real-Time Syst. (ECRTS)*, 2019, pp. 1–2.

[30] R. Gurdeep Singh and C. Scholliers, "WARDuino: A dynamic web-assembly virtual machine for programming microcontrollers," in *Proc. ACM SIGPLAN Int. Conf. Managed Program. Languages Runtimes (MPLR)*, 2019, pp. 27–36.

[31] G. Peach, R. Pan, Z. Wu, G. Parmer, C. Haster, and L. Cherkasova, "eWASM: Practical software fault isolation for reliable embedded devices," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 39, no. 11, pp. 3492–3505, 2020.

[32] N. S. Almakhdhub, A. A. Clements, S. Bagchi, and M. Payer, "µRAI: Securing embedded systems with return address integrity," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2020.

[33] T. Nyman, J.-E. Ekberg, L. Davi, and N. Asokan, "CaRE: Hardware-supported call and return enforcement for commercial microcontrollers," in *Proc. 20th Int. Symp. Res. Attacks, Intrusions, Defenses (RAID)*, Atlanta, GA, USA, Sep. 18–20, Springer, 2017, pp. 259–284.

[34] Y. Du, Z. Shen, K. Dharsee, J. Zhou, R. J. Walls, and J. Criswell, "Holistic control-flow protection on real-time embedded systems with kage," in *Proc. 31st USENIX Secur. Symp. (USENIX Secur.)*, 2022, pp. 2281–2298.

[35] W. Choi, M. Seo, S. Lee, and B. B. Kang, "SuM: Efficient shadow stack protection on ARM Cortex-M," *Comput. Secur.*, vol. 136, p. 103568, 2024.

[36] X. Tan and Z. Zhao, "Sherloc: Secure and holistic control-flow violation detection on embedded systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, 2023, pp. 1332–1346.

[37] "IAR embedded workbench." IAR. Accessed: Jun. 2, 2024. [Online]. Available: https://www.iar.com

[38] A. Lackorzynski et al., "L4linux porting optimizations," Master's thesis, TU Dresden, 2004.

[39] P. K. Gadepalli, R. Gifford, L. Baier, M. Kelly, and G. Parmer, "Temporal capabilities: Access control for time," in *Proc. 38th IEEE Real-Time Syst. Symp. (RTSS)*, 2017, pp. 56–67.

[40] B. Dragovic et al., "Xen and the art of virtualization," *ACM SIGOPS Oper. Syst. Rev.*, vol. 37, no. 5, pp. 164–177, 2003.

[41] D. Chisnall, *The Definitive Guide to the Xen Hypervisor*. Englewood Cliffs, NJ, USA: Prentice-Hall, 2008.

[42] R. Pan and G. Parmer, "MxU: Towards predictable, flexible, and efficient memory access control for the secure IoT," *ACM Trans. Embedded Comput. Syst. (TECS)*, vol. 18, no. 5s, pp. 1–20, 2019.

[43] R. Pan and G. Parmer, "SBIs: Application access to safe, baremetal interrupt latencies," in *Proc. IEEE 28th Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2022, pp. 82–94.

[44] P. K. Gadepalli, R. Pan, and G. Parmer, "Slite: OS support for near zero-cost, configurable scheduling," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*, 2020, pp. 160–173.

[45] S. Kubica and M. Kogias, "μbpf: Using eBPF for microcontroller compartmentalization," in *Proc. ACM SIGCOMM Workshop eBPF Kernel Extensions*, 2024, pp. 23–49.

Junchao Li received the bachelor's degree from Shandong University of Science and Technology, China, in 2022. He is currently working toward the master's degree with the School of Computer Science and Technology, Shandong University, China. His research interests include microcontroller systems and real-time systems.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.

Runsheng Hou received the bachelor's degree from Qingdao University, China, in 2023. He is currently working toward the master's degree with the School of Computer Science and Technology, Shandong University, China. His research interests include microkernel system, lightweight virtualization, and distributed computing.

Guangyong Shang is currently working toward the Ph.D. degree with the School of Computer Science and Engineering, Shandong University, China. He currently serves as the Chief Technology Officer and the Deputy General Manager of Inspur Yunzhou Industrial Internet Company Ltd, and is the Chief Architect of the national-level cross-industry and cross-field platform, "Yunzhou Platform." He also holds positions in the expert database of the Ministry of Science and Technology of China and the Director of the Shandong Provincial Manufacturing Innovation Center. He has long engaged in the field of digital industries, including IIoT and blockchain.

Huanle Zhang received the Ph.D. degree in computer science from the University of California, Davis, in 2020. He is an Associate Professor with the School of Computer Science and Technology, Shandong University, China. He has published more than 30 research papers, granted two US patents and multiple China patents. He won 2015 SenSys best paper award and 2019 HotMobile Demo Runner-up award. His research interests include data-centric AI, data privacy, IoT, and mobile systems.

Xiuzhen Cheng (Fellow, IEEE) received the M.S. and Ph.D. degrees in computer science from the University of Minnesota–Twin Cities, in 2000 and 2002, respectively. She is a Professor with the School of Computer Science and Technology, Shandong University, China. Her research interests include wireless and mobile security, cyber physical systems, sensor networking, and mobile computing. She worked as a Program Director for the US NSF from 2006 (full time), and from 2008 to 2010 (part time). She received the NSF CAREER Award in 2004.

Runyu Pan received the Ph.D. degree in computer science from George Washington University, in 2021. He is an Associate Professor with the School of Computer Science and Technology, Shandong University, China. He has published more than ten research papers, and has granted multiple patents. His research interests include IoT, microcontroller systems, real-time systems, and AI-based resource management.

Authorized licensed use limited to: University of L'Aquila. Downloaded on October 22,2025 at 10:23:21 UTC from IEEE Xplore. Restrictions apply.